

Where and when will the next crime be committed?

Paolo Roberto, Samuele Rivi, Luca Ferrarini

1 Introduction to the problem

The object of the study is to create and analyse a forecasting model with the aim to predict where and when crime will take place: in fact, it could be possible to identify patterns and recurring trends that might allow us to forecast the probability of new crimes with a high degree of precision. Therefore, this study aims to analyse historical crime data from a U.S. city to predict the locations and times at which crimes may occur. This approach could lead to a better allocation of the police resources that could improve the efficiency in crime prevention.

1.1 Input files

The above-mentioned data is contained in two csv files: the first one is called ***crime*** and contains all the information about each crime that took place in the city; while the second one is called ***offense_codes*** and contains the general information about the types of crimes that are classified.

These two files can be read and transformed in dataframes. The first one is ***crime***, which has these columns:

- *incident_id*: unique code that identifies the infraction;
- *offense_id*: unique code that resumes the principal information (made of: *incident_id* + *offense_code* (in 4 digits) + *offense_code_extension* (in 2));
- *offense_code*: code that stands for the crime committed;
- *offense_code_extension*: with *offense_code* specify the crime;
- *offense_type_id*: speaking code that is related to a unique couple of *offense_code* and *offense_code_extension*;
- *offense_category_id*: it is the category of the crime committed;
- *first_occurrence_date*: it is the first day of the crime committed;
- *last_occurrence_date*: if a crime lasted for more than a day it signals when the crime ended;
- *reported_date*: dates when the crime is reported;
- *incident_address*: addresses where the crime is committed;
- *geo_x*: between 1 and 40674766;
- *geo_y*: between 111 and 10890452;
- *geo_lon*: between -115.4641002 and 0;
- *geo_lat*: between 0 e 39.9009613;
- *district_id*: code that identifies the district where the crime took place;
- *precinct_id*: code that identifies the precinct where the crime took place;
- *neighborhood_id*: code that identifies something that is near the place where the crime took place;

- *is_crime*: binary code that specifies if the infraction is actually a crime;
- *is_traffic*: it is used to identify if the crime is a traffic, still it is always equal to zero, so we delete it;
- *victim_count*: number of victims of the crime.

The other one is ***offense_codes***:

- *OBJECTID*: unique code indicating the index of the dataframe;
- *OFFENSE_CODE*: the same of the other dataframe;
- *OFFENSE_CODE_EXTENSION*: the same of the other dataframe;
- *OFFENSE_TYPE_ID*: the same of the other dataframe;
- *OFFENSE_TYPE_NAME*: reports the complete name of the offense type;
- *OFFENSE_CATEGORY_ID*: the same of the other dataframe;
- *OFFENSE_CATEGORY_NAME*: reports the complete name of the category;
- *IS_CRIME*: the same of the other dataframe;
- *IS_TRAFFIC*: the same of the other dataframe.

2 Data processing

The preprocessing steps are documented in *Preprocessing_definitivo.ipynb*, which summarize all the file reading and the preprocessing conducted in this study.

2.1 Data cleaning

The original data needs an initial cleaning: there are some records in *offense_codes.csv* that contain the same values in different rows, bringing errors to some crime.csv columns, such as *type_id*, *offense_code* or *offense_code_extension* and consequentially in *offense_id*. So the most frequent records in crime.csv were found and substituted instead of the least frequent ones, then the wrong records in offense_codes.csv were deleted and the two dataframes were merged. At the end the *offense_id* was recalculated with the correct records. Also, missing values in most of the columns are handled. Moreover there was some erroneous geographical data were found in *precinct_id*, *geo_lat* and *geo_lon* and subsequently removed.

2.2 New columns

In the present case study, a critical information is time: despite the fact that there are three columns with information regarding time, it is vital to use mainly *first_occurrence_date* because it is the solely time column with only not null values. After the transformation of every time column to date time, two new columns were calculated: the first one is *hour*, which reports the time slot of the *first_occurrence_date* out of the 24 hours; the other one is *time_of_day*, which reports a more general time information, selecting the time slot between only four periods. Each period lasts 6 hours and they are divided in this way: *night* from 1 to 6; *morning* from 7 to 12; *afternoon* from 13 to 18 and *evening* from 19 to 24.

3 Creation of the Database

After all the preprocessing, the first goal is the creation of a database that can store all the data in the correct way. Here the logical schema of the database is reported in Figure 1.

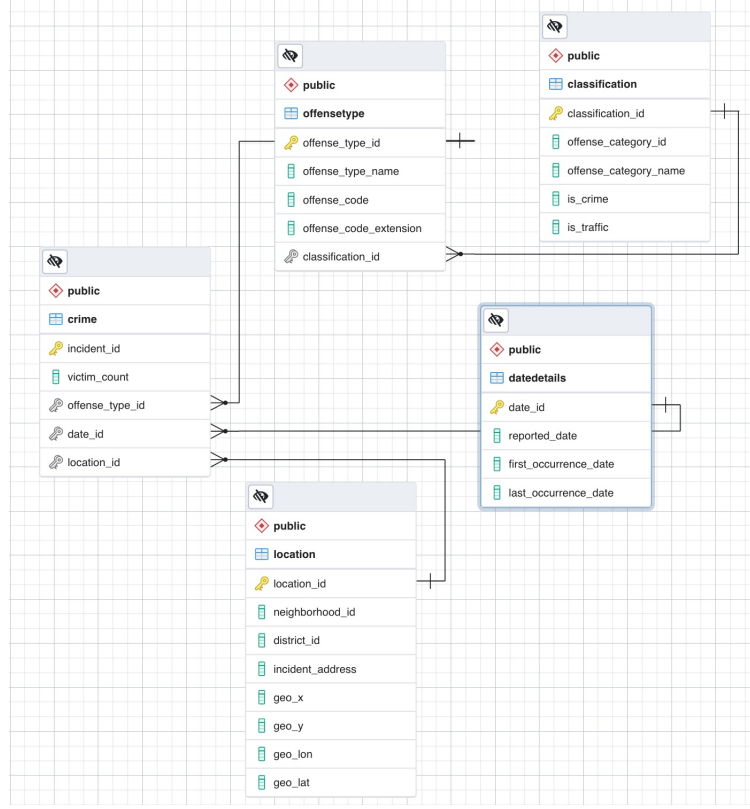


Figure 1: Logical schema

The new columns were not included because the original ones has priority in this study and also because they can easily be obtained by the original ones. The database is organised by column topic:

- *crime*: the primary key is *incident_id*, it stores *victim_count* and the reference to the other tables that store all the information
- *offensetype*: the primary key is *offense_type_id* which works as a reference to the table *crime*. The table stores the general information about the type of crime and also a reference to *classification* for more accurate classification
- *classification*: the primary key is *classification_id* which works as a reference to the table *offensetype*. The table stores the precise information about the classification of the crime
- *datedetails*: the primary key is *date_id* which works as a reference to the table *crime*. The table stores all the information about the time of the crime
- *location*: the primary key is *location_id* which works as a reference to the table *crime*. The table stores the information about the place where the crime occurs

4 Forecasting model

After the preprocessing, then the analysis and the creation of the database, it is finally possible to create a forecasting model. The goal of this model is to predict the time and location of future crimes. To achieve this, we must define these dimensions:

- *Time*: for the time forecast the new columns *hour* and *time_of_day* are just right, so both can be used depending on the required level of precision

- *Place*: for the place forecast there is much more information in the database, but the decision is made on *district_id* and *precinct_id* which seem to have the correct degree of granularity

The following are the models we have covered and their application to the case study.

4.1 Issues about the neural network

Initially, the study was addressed to the realization of a neural network, in order to forecast both time and place information simultaneously. Here the characteristics of the neural network are reported.

The network presents an input layer, two shared layers (the first with 256 and the second with 128 neurons) that use ReLU activation and two output nodes, both dense layers with softmax activation to predict time and place classes. To mitigate over-fitting we employ early stopping with patience in order to have a more generalized training process. In the neural network, *adam* is used as optimizer, *categorical_crossentropy* is used as loss function to both the output predicted. The time is more weighted because it is discovered to be less accurate on the prediction.

The next step was to use this model to forecast, so we create a list of columns as an input to the model: *offense_type_id*, *neighborhood_id*, *victim_count*, *offense_code*, *offense_code_extension*, *offense_category_id*, *is_crime*, *is_traffic*. Some of these columns needed to be separated using a *.getdummies* in order to contain values that can be used by the network. The network is used to predict hour and time_of_day for time dimension and district, precinct and cluster (which will be better specified later in the exposition) for place dimension, bringing to this results:

Place dimension	Place accuracy	Time dimension	Time accuracy	Loss
Precinct	88%	Hour	10%	over 2
District	96%	Time of day	39%	over 1.2
Cluster	79%	Time of day	38%	over 0.8

Table 1: Results of the Neural Networks

The low accuracy value on hour as time dimension obtained in precinct prediction brings to use time of day as time dimension.

Due to these results, we choose to have a major focus on the difference between the accuracy on the place and on the time dimension, so we noticed that the column *neighborhood_id* is an other key place dimension, including it in the input does not yield interesting forecasts, but removing it leads to the following results:

Place dimension	Place accuracy	Time dimension	Time accuracy	Loss
Precinct	8%	Hour	9%	over 3
District	29%	Time of day	36%	over 3
Cluster	15%	Time of day	36%	over 2.3

Table 2: Results of the Neural Networks without *neighborhood_id*

After the observation of the low quality of the results, we have chosen to try other forecasting models.

Anyway, the neural network that we produced are reported in three different files: *District_id.ipynb*, *Precinct_id.ipynb* and *Cluster_id.ipynb*.

4.2 Random Forest

Due to these issues, we attempt a different approach by implementing a Random Forest model: we deploy a Random Forest with 100 estimators, minimum sample split 5 and a fixed random state in order to prevent overfitting. The Random Forest allows to deal with non-linearity and complex relationship and prevent overfitting by averaging of trees and using minimum sample split that forces the model to make splits only when necessary. Another important thing added is the balancing of the test set based on the forecasted dimension for each single model, having a number

of rows similar to the complete dataset (approximately 350000), with a code similar to the one we can see in Figure 2. We separate the five produced Forests in two sets:

- The first one is the one that forecasts time dimensions, which contains hour and time_of_day ones. These two have the same columns as input: offense_id, offense_type_id, incident_address, geo_x, geo_y, geo_lon, geo_lat, district_id, precinct_id, neighborhood_id, victim_count, offense_code, offense_code_extension, offense_category_id, is_crime, is_traffic. These are all the columns that do not contain any time related information.
- The other one is the one that contains forecasts about place dimensions, such as precinct_id, district_id and cluster_id. These three have the same columns as input: offense_id, offense_type_id, victim_count, offense_code, offense_code_extension, offense_category_id, is_crime, is_traffic, hour, time_of_day, first_year, first_month, first_day, last_year, last_month, last_day, reported_year, reported_month, reported_day. These are all the columns that do not contain any place related information.

This type of model is simpler to implement, but it needs a single version for every dimension we want to forecast.

```
# bilanciamento district
threshold = 0
district_counts = df_original['district_id'].value_counts()
classi_validi = district_counts[district_counts >= threshold].index
df_balanced = df_original[df_original['district_id'].isin(classi_validi)]

balanced_data = []
for district in classi_validi:
    class_data = df_balanced[df_balanced['district_id'] == district]
    balanced_class = resample(class_data, replace=True, n_samples=50000, random_state=42)
    balanced_data.append(balanced_class)

df_balanced = pd.concat(balanced_data)

df = df_balanced.copy()
```

Figure 2: Balancing training set code

4.3 Precinct

The first forecast, concerns the *precinct_id*, that divides the city in 40 zones, which distributions are shown in the following figure. This study allows a more punctual place forecast.

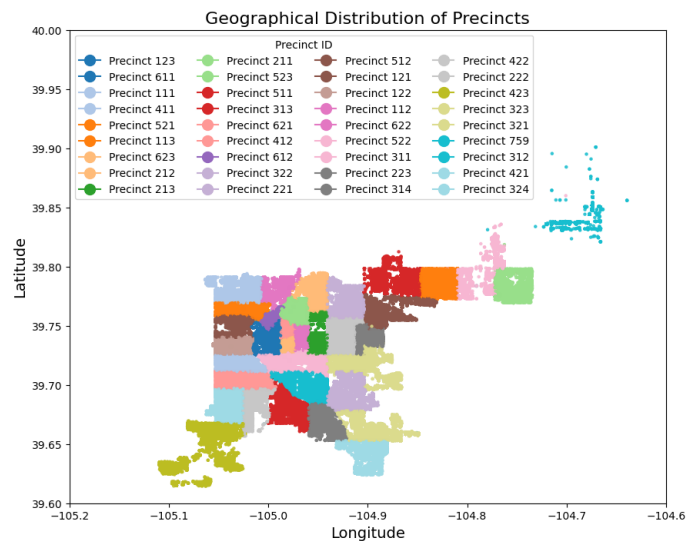


Figure 3: Valid precinct distribution

4.3.1 District

Another place forecast, the most generic one, concerns the *district_id*, that divides the city in only 7 zones, which distributions are shown in the following figure.

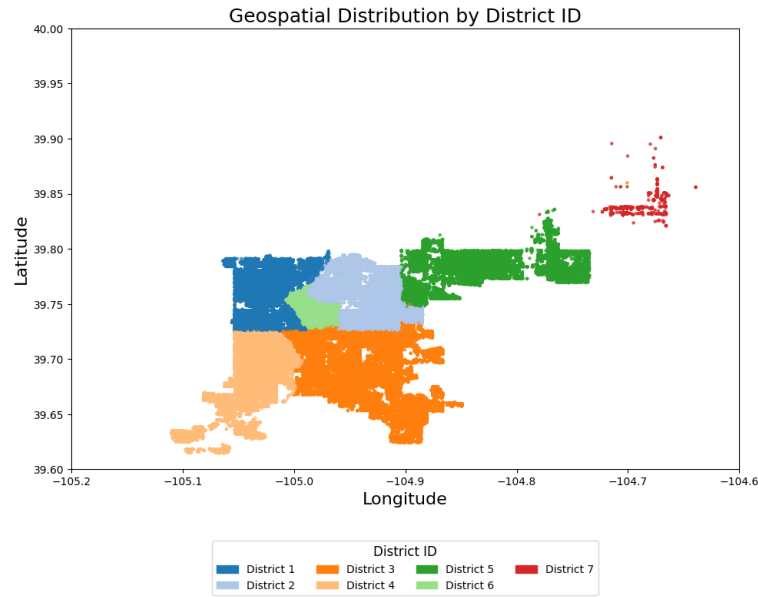


Figure 4: Valid district distribution

4.4 Cluster

In order to have an half point between a too generic and a too punctual place distribution, we decide to make a subdivision of all the infraction between 20 clustering using a K-means algorithm on *geo.lat* and *geo.lon*.

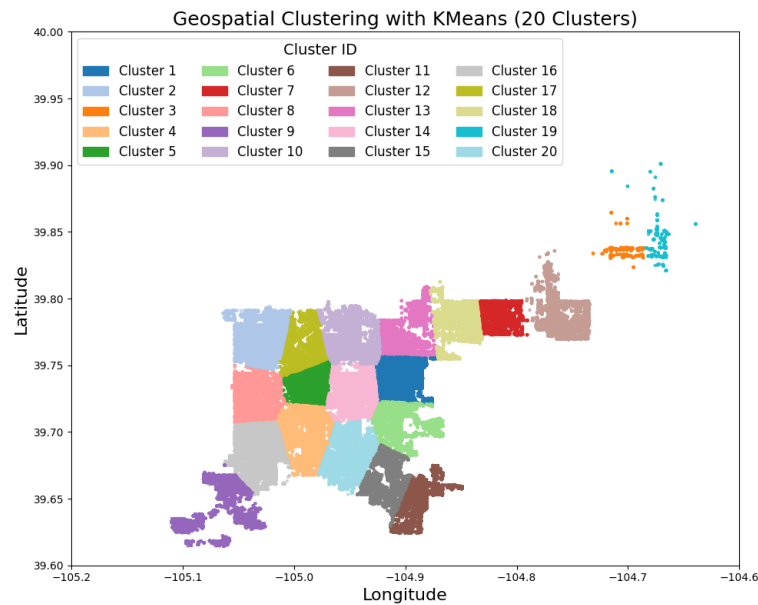


Figure 5: Valid cluster distribution

4.5 Hour

For time forecast we use at first hour which is a more specific feature.

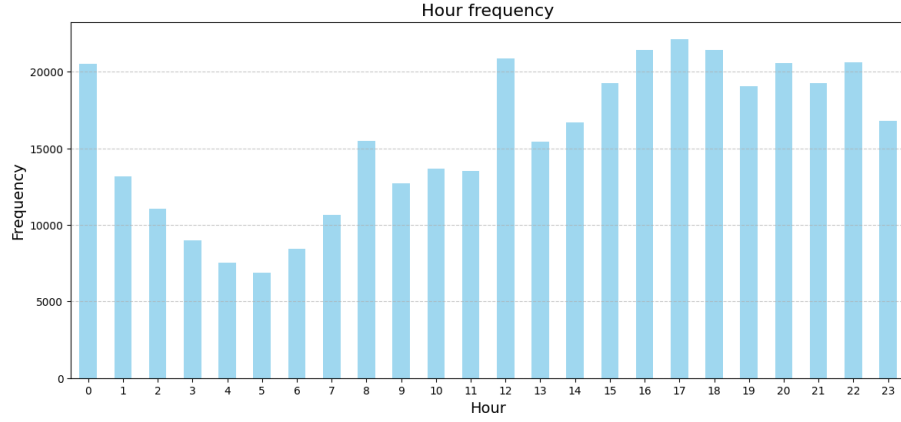


Figure 6: Hour distribution

4.6 Time of day

Last but not least we forecast the time as time of day, which is the most generic feature.

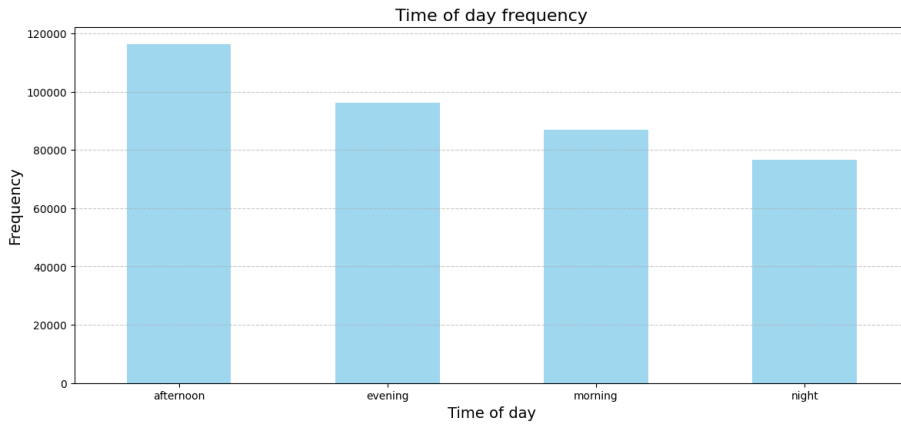


Figure 7: Time of day distribution

5 Results

Finally, we present the results obtained from the Random Forest models developed in this study:

Forecast	Accuracy	Avg Precision	Avg Recall	Avg F1
Precinct	57%	56%	57%	57%
District	65%	64%	65%	64%
Cluster	63%	62%	63%	62%
Hour	55%	55%	55%	55%
Time of day	67%	67%	67%	67%

Table 3: Results of the Random Forests

If we consider also the neighborhood_id, in order to make a better comparison with the previous model, we obtain the results:

Forecast	Accuracy	Avg Precision	Avg Recall	Avg F1
Precinct	85%	85%	85%	85%
District	90%	90%	90%	90%
Cluster	86%	86%	86%	86%

Table 4: Results of the Random Forests with neighborhood_id

Because we search correlations between all the not time related information and all time related one, the results about Hour and Time of day do not change, so we do not report time dimension results.

All the code produced for the Random Forests is contained in the file named *Random_forest.ipynb*. As we can see in that file, the balancing combined to the different distribution of each dimension (Figure 3 to 7) lead to a high differences in the forecast metrics calculated on the single classes, but we decide to maintain that structure in order to preserve complexity and variability of the data.

6 Conclusions

Ultimately, our findings indicate that it is not possible to predict both time and location simultaneously, as demonstrated by the neural network results. Thus, we conclude that no strong correlation exists between general crime information and its temporal or spatial dimensions. The situation is different in relation to the Random Forests: in fact, we could understand that if only consider one single dimension between time and place one, as long as the not forecasted one is given as input, it is possible to find a correlation between all the information that does not concern the time and the time dimension, and otherwise, between all the information that does not concern the place and the place dimension.

However, as we can you from the metrics, these correlations are not so strong and they might be due to overfitting, although we have tried to avoid it. In conclusion, we cannot accurately forecast where and when a crime will be committed, but we could use this study as an interesting first step into this case.